

Flock Energy — Engineering Take-Home

Welcome, and thanks for taking the time. We've tried to make this feel like a **real task from our backlog** rather than a puzzle. There are no trick questions and no single "correct" solution — we care far more about how you think than about how much you finish.

Time: aim for **4–6 hours**. A focused, well-reasoned partial solution beats an exhausted "complete" one. If you find yourself over budget, stop and write about what you'd do next — that's worth real credit.

Background

Flock works with electricity distribution utilities. One utility runs an ageing internal web portal — "**Urja Meter Ops**" — that their field and operations staff use to look up smart-meter information: meter details, where a meter sits in the network, and its recent consumption.

The portal works fine for humans clicking around. The problem: **it has no API**. Product and data teams keep asking for programmatic access, and right now the only "integration" is people copy-pasting from the browser. We've solved this kind of thing before by building a clean service in front of a legacy system — and that's what we'd like you to do here.

Your task

Build a clean, documented API service over the Urja Meter Ops portal.

You have a login to a running instance of the portal. Explore it, work out how it actually works under the hood, and build a service that exposes its data in a sensible, well-structured way that another engineer (or program) could consume without ever touching the original portal.

Access:

Portal URL	https://urja-ops.flockenergy.tech
Username	operator@urja.local
Password	urja-ops-2026

That's deliberately most of what we're going to tell you. Part of the exercise is figuring out the rest. Where something is unclear or unspecified, **make a reasonable decision and write down why** — we read those notes closely.

Ground rules

- **Any language, any stack, any libraries.** Use whatever you're most effective in. We're not evaluating your choice of framework.
- **AI assistants are allowed — for guidance, not to do the work for you.** Using an LLM to explain a concept, unblock an error, or review your code is completely fine and encouraged. But the core investigation has to be *yours*: an assistant can't see what we've given you, and it will show in the

follow-up conversation if the understanding isn't real.

- **No paid services required.** Everything can be built and run locally.
- Treat the portal as read-only. You don't need to (and shouldn't try to) attack or break it — everything you need to use it is available to a normal logged-in user.

What we're evaluating

Roughly in this order:

1. **Problem solving** — how you break down an unfamiliar system and make progress when the path isn't given to you.
2. **Engineering judgment** — the trade-offs you make, what you choose to build vs. skip, and whether the result is appropriately sized (neither over- nor under-engineered).
3. **Learning ability** — how you teach yourself the things you need along the way.
4. **Communication** — how clearly you explain what you built, what you assumed, and why.
5. **Code quality** — readable, sensibly organised, maintainable.
6. **Architecture, testing, speed** — in that order, and less important than the above.

You will not be marked down for not knowing a specific tool, and you will not win points for using a fancy one.

Deliverables

Submit a **public GitHub repository** containing:

1. **Source code** for your API service.
2. **README** — the front door to your submission. We should be able to understand, run, and evaluate everything from here. It must contain (or clearly link to) all of: what you built and how it's structured, how to install/configure/run it, a sample request, your assumptions, design decisions & trade-offs, what you intentionally skipped, what you'd improve with more time, and where to find **PROTOCOL.md** and **openapi.json**. Assume the reader has zero prior context.
3. **PROTOCOL.md** — a short write-up of *how the portal actually works*, as you discovered it: what data is available, how you get to it, how access/auth works, and any quirks or surprises you found. This is one of the most valuable things you'll submit.
4. **openapi.json** — an OpenAPI (3.x) description of **your** API — the clean one you're building, *not* the portal's. How you produce it is up to you (by hand, from code annotations, or a generator). Optionally, serve a rendered view of it (e.g. Scalar, Swagger UI, or Redoc) so it can be explored in a browser.
5. **Notes** (can live in the README) covering:
 - Assumptions you made.
 - Design decisions and trade-offs.
 - What you intentionally left out, and why.
 - What you'd improve or do differently with more time.
6. **Reflection** — the questions below, answered in your README or a linked **REFLECTION.md**.

Reflection

Put your answers in your **README** (a **## Reflection** section) or in a **REFLECTION.md** linked from the README. A paragraph each is plenty:

- What assumptions did you make?

- Which part was the most difficult, and how did you get unstuck?
- If you had another day, what would you improve?
- What mistake did you make while solving this (there's always one)?
- If you were reviewing your own submission, what would you criticise?

Optional extensions (only if you have time and curiosity)

These are **not required**. They're here so that if you finish the core and want to go deeper, there's somewhere to go. More thinking, not necessarily more code:

- **Network hierarchy.** The portal organises meters into a hierarchy. Reconstruct that structure and expose it in a useful way. Watch out for messy or inconsistent data.
- **A local index / query layer.** Make it possible to answer questions the portal itself can't easily answer (e.g. filtering across attributes, or "what's near this location?"). Include a note on *at what data scale your approach would start to struggle, and what you'd change*.
- **Freshness / caching.** If you cache anything, how do you decide when it's stale? -
- **Robustness.** What happens to your client over a long-running session, or when the portal misbehaves? Handle it gracefully.
- **The full dataset.** There may be more than one way to get data out of the portal. If a bulk path exists, using it well is a nice touch.
- **A modern web client.** Build a small, modern web app on top of *your* API that presents the portal's data more usefully and attractively than the original — think creative, genuinely useful visuals (consumption charts, a map of meters, a hierarchy explorer, health or anomaly summaries — your call). We're interested in your product sense and how you present data, not pixel-perfect polish.

Submitting

- Push your work to a **public GitHub repository**.
- **Email us the link** (just reply to the email you received this assignment from). - Make sure the repository's **README contains all the relevant information** — see the README deliverable above. A reviewer should be able to clone the repo and get everything they need from the README alone, without asking you a single question.

We'll then have a short follow-up conversation where you walk us through your thinking — so optimise your write-up for *"could a teammate understand this and pick it up?"*

Have fun with it. We did.